



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH



**Barcelona
Supercomputing
Center**
Centro Nacional de Supercomputación

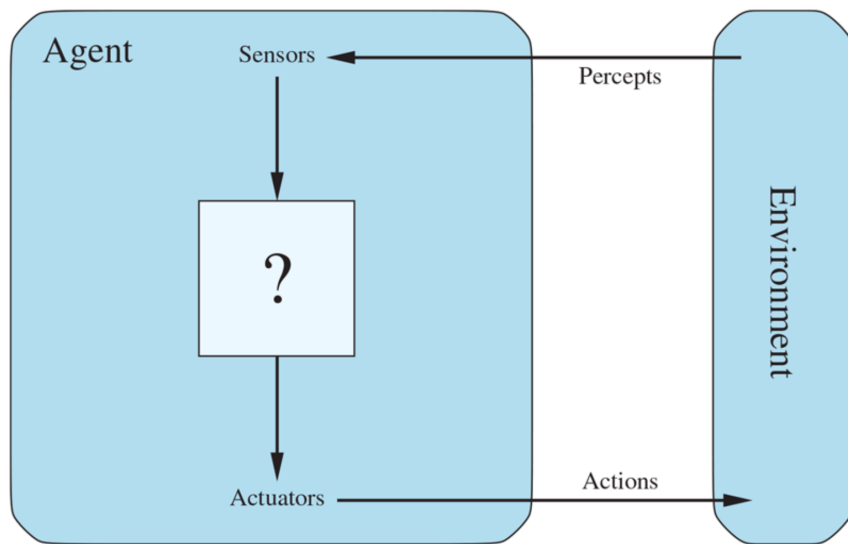


Aprendizaje por refuerzo

Breve introducción

Agente y entorno

- Un agente es un sistema computacional capaz de actuar en un entorno, de manera autónoma, para cumplir los objetivos de diseño
 - Guiado por lógica o reglas simbólicas, o
 - **Guiado por una función de utilidad subsimbólica**
- A través de sensores, el agente recibe **percepciones** del entorno
- El agente **actúa** en el entorno a través de sus actuadores



Señal de recompensa

- ❖ Señal (o función) que guía al agente basándose en:
 - el estado en el que estaba,
 - la acción que ha tomado y
 - el estado al que llega

$$\mathcal{R}: S \times A \times S \rightarrow \mathbb{R}$$

- ❖ La función de utilidad se puede calcular a partir de esta señal
 - Habitualmente: el valor esperado de la utilidad es una función sobre la suma de recompensas recibidas durante la ejecución
- ❖ ¿De dónde viene esta señal? Generalmente, del entorno

La hipótesis de recompensa

*That all of **what we mean by goals and purposes** can be well thought of as **maximization of the expected value of the cumulative sum of a received scalar signal** (reward).*

The reward hypothesis, Richard Sutton

<http://incompleteideas.net/rlai.cs.ualberta.ca/RLAI/rewardhypothesis.html>

Problemas de decisión secuencial



Ejemplo: dados

- CS221 - Stanford



Para cada turno $t = 1, 2, \dots$

- Escoge: **jugar** o **parar**
- Efecto de **jugar**:
 - Recibes 4 euros
 - Lanzas un dado de 6 caras
 - Si el resultado es 1 o 2, se acaba el juego
 - Si el resultado es otro, se avanza al siguiente turno
- Efecto de **parar**:
 - Recibes 10 euros
 - Se acaba el juego

Formalización: MDPs

Un proceso de decisión de Markov (MDP) se define como una tupla

$\langle S, A, \mathcal{R}, \mathcal{T}, \mu, \gamma \rangle$ tal que:

S es el conjunto de estados, del cual $\bar{S} \subset S$ es el conjunto de estados finales

A es el conjunto de acciones

$\mathcal{R}: S \times A \times S \rightarrow \mathbb{R}$ es la función de recompensa, normalmente formulada como $r(s, a, s')$

$\mathcal{T}: S \times A \times S \rightarrow [0,1]$ es la función de probabilidad de transición entre estados, normalmente formulada como $p(s'|s, a)$

$\mu: S \rightarrow [0,1]$ es la función de distribución del estado inicial

$\gamma \in [0,1]$ es el factor de descuento

Problemas de decisión secuencial

❖ Ejemplo: dados

- CS221 - Stanford

❖ Para cada turno $t = 1, 2, \dots$

- Escoge: **jugar** o **parar**

- Efecto de **jugar**:

- Recibes 4 euros
- Lanzas un dado de 6 caras
 - Si el resultado es 1 o 2, se acaba el juego
 - Si el resultado es otro, se avanza al siguiente turno

- Efecto de **parar**:

- Recibes 10 euros
- Se acaba el juego

$$S = \{\text{jugando}, \text{fin}\}, \bar{S} = \{\text{fin}\}, \mu(\text{jugando}) = 1$$

$$A = \{\text{jugar}, \text{parar}\}$$

$$p(\mathbf{s} = \text{jugando} \mid \mathbf{s} = \text{jugando}, \mathbf{a} = \text{jugar}) = \frac{2}{3}$$

$$p(\text{fin} \mid \text{jugando}, \text{jugar}) = \frac{1}{3}$$

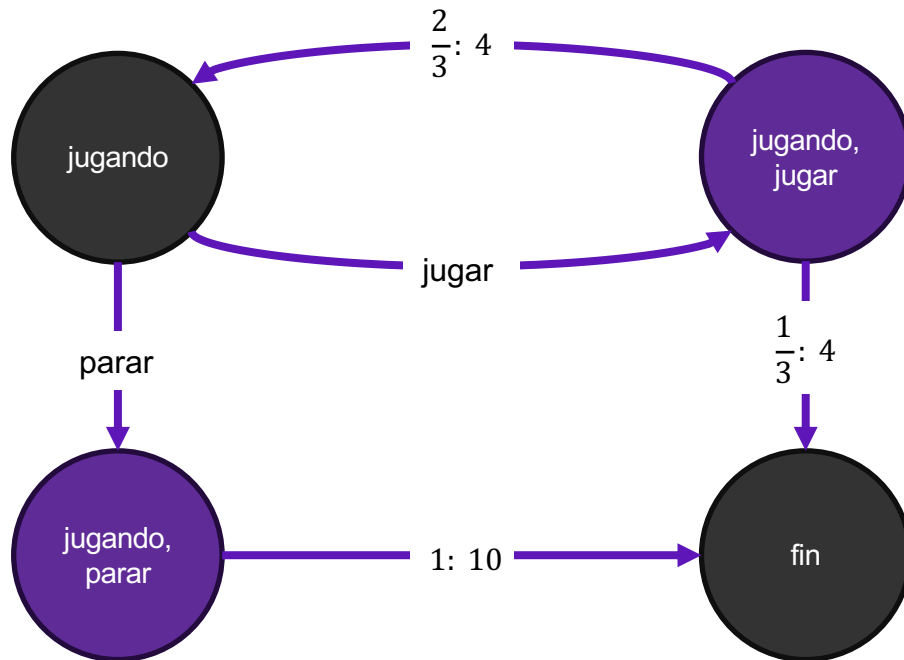
$$p(\text{fin} \mid \text{jugando}, \text{parar}) = 1$$

$$\mathcal{R}(\mathbf{s} = \text{jugando}, \mathbf{a} = \text{jugar}, \mathbf{s}' = \text{jugando}) = 4$$

$$\mathcal{R}(\text{jugando}, \text{jugar}, \text{fin}) = 4$$

$$\mathcal{R}(\text{jugando}, \text{parar}, \text{fin}) = 10$$

MDP: juego de los dados



Política

- ❖ Una solución a un MDP se llama **política** y se define como:

$$\pi: A \times S \rightarrow [0,1]$$

- ❖ Una política es una asignación, para cada par acción $a \in A$, estado $s \in S \setminus \bar{S}$, de una probabilidad $\pi(a|s)$:
 - **1 si a es la opción escogida** cuando el agente está en el estado s ,
 - **0 en cualquier otro caso**, i.e. para todas las acciones no escogidas en el estado s

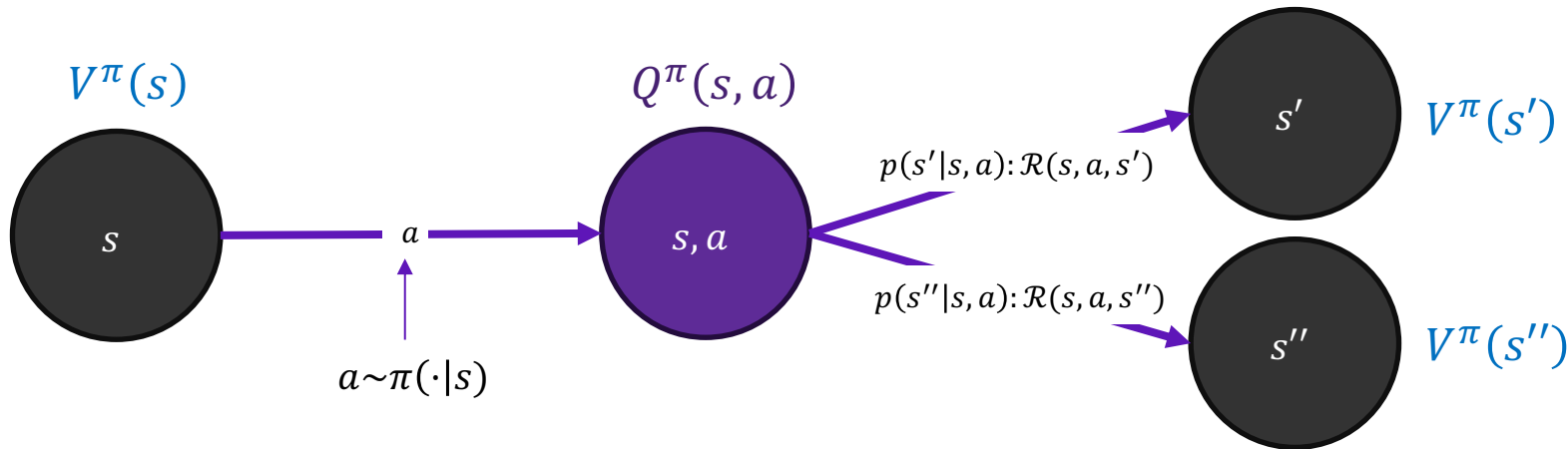
Valor de estado, valor de acción

❖ **Valor de un estado** bajo una política: $V^\pi(s)$

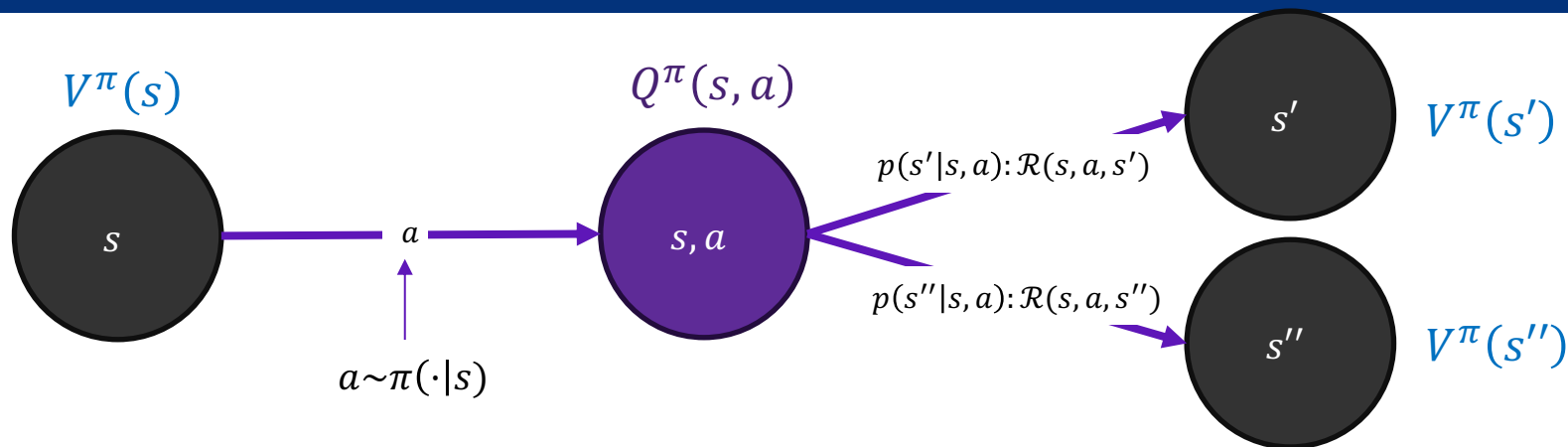
- Utilidad esperada al seguir la política π desde el estado s

❖ **Valor de una acción en un estado** bajo una política: $Q^\pi(s, a)$

- Utilidad esperada al seguir la política π tomando la acción a desde el estado s



Valor de estado, valor de acción

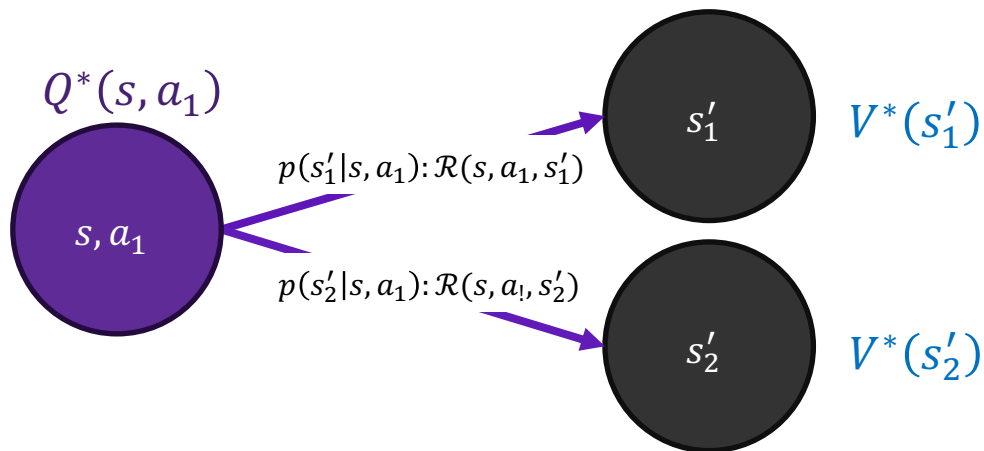


$$Q^\pi(s, a) = \sum_{s' \in \mathcal{S}} p(s' | s, a) [\mathcal{R}(s, a, s') + \gamma V^\pi(s)]$$

$$V^\pi(s) = \begin{cases} 0 & \text{si } s \in \bar{\mathcal{S}} \\ Q^\pi(s, a) & \text{en cualquier otro caso, donde } a \sim \pi(\cdot | s) \end{cases}$$

Valor acción-estado óptimo

$$Q^*(s, a) = \sum_{s' \in \mathcal{S}} p(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V^*(s')]$$
$$V^*(s) = \max_{a \in A} Q^*(s, a)$$



Política óptima

La política óptima se puede obtener simplemente escogiendo la acción $a \in A$ que maximiza la función $Q^*(s, a)$ para el estado s :

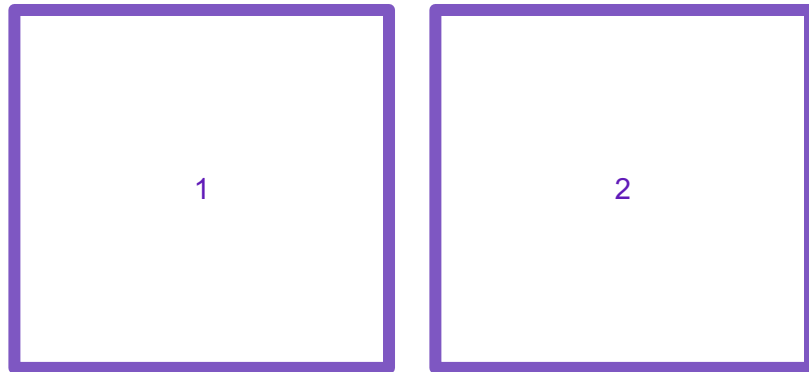
$$\pi^*(a|s) = \begin{cases} 1 & \text{si } a = \arg \max_{a' \in A} Q^*(s, a') \\ 0 & \text{en cualquier otro caso} \end{cases}$$

Esta fórmula es equivalente, por substitución, a:

$$\pi^*(a|s) = \begin{cases} 1 & \text{si } a = \arg \max_{a' \in A} \sum_{s' \in S} p(s'|s, a') [\mathcal{R}(s, a', s') + \gamma V^*(s')] \\ 0 & \text{en cualquier otro caso} \end{cases}$$

Vamos a jugar a otro juego

- Utilidad $u_0 = 0$
- Para cada turno $t = 1, 2, \dots, 20$
 - Escoge: **jugar1**, **jugar2** o **parar**
 - Efecto de **parar**:
 - Se acaba el juego con utilidad u_{t-1}
 - Efecto de **jugarX**:
 - Se asigna un valor u_t



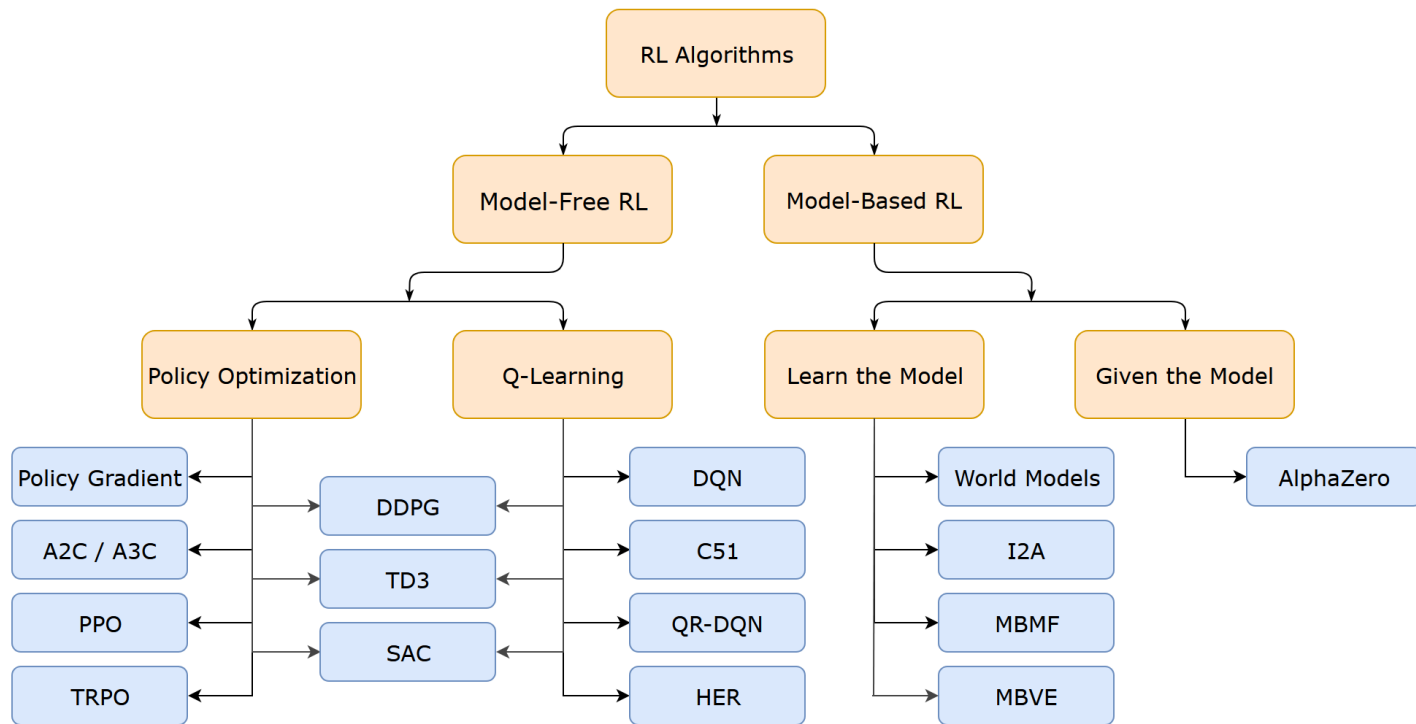
Multi-armed bandits

- ❖ ¿Cómo podemos abordar este tipo de tarea?
 - Intuición: **aprender** la ganancia esperada con cada decisión (estado-acción)
- ❖ El dilema: **exploración vs explotación**

Contexto

- ❖ Supongamos las siguientes premisas:
 - Tenemos una representación del estado
 - Tenemos el conjunto de acciones disponibles
 - No tenemos acceso a:
 - La función de recompensa, o
 - La función de transición entre estados a través de pares estado-acción
- ❖ Básicamente, no tenemos un MDP completo
 - El agente está *dentro* del MDP, pero no tiene total acceso a él
- ❖ El **aprendizaje por refuerzo** es el área que se encarga de solucionar problemas formulados de esta manera

Una taxonomía



https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html

Aprendizaje por diferencias temporales

- ❖ El aprendizaje por diferencias temporales (*Temporal Difference* o *TD Learning*) es la base de muchos métodos como SARSA o Q-learning
- ❖ Idea principal: aprender de la experiencia a partir de π

- Generar muestras de estados, acciones y recompensas: $s, \pi(s), s', r$

$$\mathcal{X} = \mathcal{R}(s, \pi(s), s') + \gamma V^\pi(s')$$

- Y actualizar $V^\pi(s)$ de manera constante, aplicando una media móvil:

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha(\mathcal{X} - V^\pi(s))$$

donde $\alpha \in (0,1]$ es la tasa de aprendizaje o *learning rate*

Algoritmo Q-learning (ϵ -greedy)

❖ Inicialización

- Parámetros: tasa de aprendizaje $\alpha \in (0,1]$, coeficiente de exploración $\epsilon > 0$
- $Q(s, a) = 0 \quad \forall s \in S, a \in A$

❖ Para cada episodio:

- Observar estado s_0
- Para cada $t = 0, 1, 2, \dots, t_{MAX}$
 - Con probabilidad ϵ :
 - Ejecutar acción aleatoria $a_t \in A$
 - Si no, ejecutar acción $a_t \in \arg \max_{a \in A} Q(s_t, a)$
 - Observar estado s_{t+1} y recompensa r_t
 - $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[\mathcal{R}(s_t, a_t, s_{t+1}) + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$

REINFORCE

- ❖ El algoritmo REINFORCE (Williams, 1992) utiliza una función de pérdida para implementar un gradiente de política que tiene en cuenta la trayectoria $[s_0, a_0, r_0, \dots, s_T, a_T, r_T]$ de un episodio:

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=0}^{T-1} \left(\sum_{t'=t}^{T-1} \gamma^{t'-t} \mathcal{R}(s_{t'}, a_{t'}, s_{t'+1}) \right) \log \pi(a_t | s_t; \theta)$$

- ❖ Observad que no necesitamos:
 - Función de transición entre estados y pares estado-acción
 - Cálculos intermedios de las funciones de valor V o Q

El algoritmo REINFORCE

❖ Inicialización:

- Función J con parámetros iniciales θ
- Política π , derivada de $J(\theta)$

❖ Para cada episodio:

- Para cada $t = 1, 2, \dots, T - 1$:
 - Observar estado s_t
 - Ejecutar acción $a_t \sim \pi(\cdot | s_t; \theta)$
 - Observar recompensa r_t , estado s_{t+1}
- Calcular pérdida $\mathcal{L}(\theta)$
- Actualizar parámetros θ , minimizando $\mathcal{L}(\theta)$

Algunos algoritmos

- ❖ **Deep Q-Network (DQN; Mnih et al, 2013):** extensión de Q-learning con redes neuronales para aproximar la función Q a partir de un buffer de experiencia (*replay buffer*)
- ❖ **Proximal Policy Optimization (PPO; Schulman et al, 2017):** evolución de ciertos algoritmos de gradiente de política como REINFORCE, con ciertas optimizaciones que permiten una actualización de política más estable, fiable y eficiente
- ❖ **Algoritmos Actor-Critic (A3C, A2C, SAC, DDPG):** híbridos en el sentido de que combinan un método *policy-based* que propone acciones (*actor*) y un método *value-based* que evalúa las acciones (*critic*) que se retroalimentan
- ❖ **AlphaZero (Silver et al, 2017):** algoritmo auto-supervisado basado en modelo (búsqueda Monte Carlo) que no necesita datos históricos, combinando la búsqueda Monte Carlo con representaciones intermedias usando redes neuronales profundas que modelan valor y política